

Experiment -1

```
import gensim.downloader as api
model=api.load("glove-wiki-gigaword-50")
print(model.most_similar(positive=['walking','swim'],negative=['walk']))
result=model.most_similar(positive=['king','woman'],negative=['man'])
print(f"'king'+ 'man' - 'woman' is closest to:{result[0]}")

[('swimming', 0.8071774244308472), ('swimmers', 0.7504438757896423), ('swims', 0.7454535365104675), ('surfing', 0.7409682273864746), ('biking', 0.736449122428894), ('swam', 0.7311382293701172), ('jumping', 0.7218191027641296), ('diving', 0.7187682390213013), ('kayaking', 0.7121445536613464), ('snowboarding', 0.7015390396118164)]
'king'+ 'man' - 'woman' is closest to:('queen', 0.8523604273796082)
```

Experiment-2

```
import gensim.downloader as api
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

# Load model once
model = api.load("glove-wiki-gigaword-50")

words = ['queen','throne','prince','daughter','elizabeth','princess','kingdom','morning','crown','castle','throne','prince','daughter','elizabeth','princess','kingdom','morning','crown','castle']
X = np.array([model[w] for w in words])

# Reduce dimensions
pca = PCA(2).fit_transform(X)
tsne = TSNE(2, perplexity=5, max_iter=250, random_state=42).fit_transform(X)

# Plot
fig = make_subplots(rows=1, cols=2, subplot_titles=("PCA", "t-SNE"))

fig.add_trace(go.Scatter(x=pca[:,0], y=pca[:,1], mode='markers+text', text=words),
fig.add_trace(go.Scatter(x=tsne[:,0], y=tsne[:,1], mode='markers+text', text=words))

fig.show()
```

Experiment-3

```
from gensim.models import Word2Vec
from gensim.utils import simple_preprocess
sentences=["This is a legal document about contracts.",
"The court will review the legal case.",
"Medical professionals required specific training.",
"This is a medical report about the patient."]
tokenized_sentences=[simple_preprocess(sentence) for sentence in sentences]
model=Word2Vec(sentences=tokenized_sentences,vector_size=50,min_count=1)
print(model.wv.most_similar("legal"))
```

```
[('about', 0.1655503511428833), ('review', 0.1552918553352356), ('court', 0.1438906192779541), ('required', 0.13939765095710754), ('the', 0.12674154341220856), ('will', 0.12120247632265091), ('contracts', 0.10523671656847), ('report', 0.08873622864484787), ('specific', 0.03234238550066948), ('professionals', 0.02049289084970951)]
```

Experiment-4

```
#!/pip install transformers gensim nltk

import gensim.downloader as api
from transformers import pipeline
import nltk
nltk.download('punkt')

wv = api.load("glove-wiki-gigaword-100")
gen = pipeline("text-generation", model="gpt2")

p = "who is king"
print("Original:", p)

# Replace "king"
try:
    p2 = p.replace("king", wv.most_similar("king")[0][0])
except:
    p2 = p

print("Enriched:", p2)

# Generate
o1 = gen(p, max_length=100)[0]['generated_text']
o2 = gen(p2, max_length=100)[0]['generated_text']

print("\nOriginal:\n", o1)
print("\nEnriched:\n", o2)

print("\nLength:", len(o1), len(o2))
print("Detail:", o1.count("."), o2.count("."))
```

```
[nltk_data] Downloading package punkt to /home/mckb/nltk_data...
[nltk_data] Package punkt is already up-to-date!
Loading weights: 100%|██████████| 148/148 [00:00<00:00, 11832.51it/s]
[transformers] Passing `generation_config` together with generation-related arguments=({'max_length'}) is deprecated and will be removed in future versions. Please pass either a `generation_config` object OR all generation parameters explicitly, but not both.
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
[transformers] Both `max_new_tokens` (=256) and `max_length` (=100) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Original: who is king
Enriched: who is prince
```

```
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer GPT2Tokenizer. The clean_up_tokenization post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokeni
```


The temple of Solomon was built in the city of Jerusalem, which was called The Temple of Solomon.

The Temple of Solomon was built in the city of Jerusalem, which was called The Temple of Solomon

Length: 1036 1139

Detail: 3 11

Experiment-5

```
import gensim.downloader as api

# Load model
model = api.load("glove-wiki-gigaword-50")

# Input seed word
seed = input("Enter a word: ")

# Get similar words
similar = [w[0] for w in model.most_similar(seed, topn=5)]

# Create paragraph
paragraph = f"""
The {seed} was known for its connection with {similar[0]} and {similar[1]}.
People often related it to {similar[2]} and {similar[3]}.
In many stories, {similar[4]} played an important role alongside the {seed}.
"""

print("\nGenerated Paragraph:\n")
print(paragraph)
```

Generated Paragraph:

The king was known for its connection with prince and queen.
People often related it to ii and emperor.
In many stories, son played an important role alongside the king.

Experiment-6

```
from transformers import pipeline

# Load sentiment model
analyzer = pipeline("sentiment-analysis")

# Sample inputs (real-world: customer reviews)
reviews = [
    "The product is amazing! I love it.",
    "I'm very disappointed. The service was terrible.",
    "It was an average experience.",
    "Absolutely fantastic quality!",
    "Not great, but okay."
]
```

```
# Analyze sentiment
for r in reviews:
    result = analyzer(r)[0]
    print(f"{r} → {result['label']} ({result['score']:.2f})")
```

```
/home/mckb/Git/College/GenAI/.venv/lib/python3.14/site-packages/tqdm/auto.py:21: T
qdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See http
s://ipywidgets.readthedocs.io/en/stable/user_install.html
    from .autonotebook import tqdm as notebook_tqdm
[transformers] No model was supplied, defaulted to distilbert/distilbert-base-unca
sed-finetuned-sst-2-english and revision 714eb0f.
Using a pipeline without specifying a model name and revision in production is not
recommended.
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_T
OKEN to enable higher rate limits and faster downloads.
```

Experiment-7

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

text = """
Summarize this:
Artificial Intelligence (AI) is a rapidly evolving field focused on creating intell
It has impacted industries like healthcare, finance, and education. AI applications
self-driving cars, and recommendation systems. However, it also raises ethical conc
and job displacement.
"""

output = generator(text, max_length=10, num_return_sequences=1)

print(output[0]['generated_text'])
```

```
Loading weights: 100%|██████████| 148/148 [00:00<00:00, 10411.54it/s]
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for open-end generat
ion.
[transformers] Both `max_new_tokens` (=256) and `max_length` (=10) seem to have bee
n set. `max_new_tokens` will take precedence. Please refer to the documentation fo
r more information. (https://huggingface.co/docs/transformers/main/en/main_classe
s/text_generation)
```

```
Summarize this:
Artificial Intelligence (AI) is a rapidly evolving field focused on creating intel
ligent machines.
It has impacted industries like healthcare, finance, and education. AI application
s include chatbots,
self-driving cars, and recommendation systems. However, it also raises ethical con
cerns like bias
and job displacement.
```

has a great deal of potential, but the problem is that it is still in its infanc
y. The best way to learn is to learn from other programmers. We still have a long
way to go before we can learn what it is like to be human.

When I think of AI, I think of the people who are most likely to have some sort of
programming skills. The ones who are most likely to have a degree of skill in AI.
In contrast, the people who are most likely to have no programming skills, at leas
t as a whole.

We are in the early stages of human-computer interaction and we have the potential to develop complex artificial intelligence technologies. We are not sure what those technologies will be. But we are definitely in the process of creating a computer that can learn a lot from the human race, from the world around us. To me, that's a great step forward for humanity.

We are very excited about this, and we are also very excited about how we can help create the future.

When did you first begin thinking about AI?

I was very early in my career, and as I got older I got a lot more inspired by natural language processing. I saw that there was something interesting about natural

Experiment-9

```
import json

data = json.load(open("institutions.json"))

name = input("Enter institution name: ").strip()

if name in data:
    d = data[name]
    print(
        f"Founder: {d.get('founder', 'N/A')}\n"
        f"Founded: {d.get('founded', 'N/A')}\n"
        f"Branches: {' , '.join(d.get('branches', [])) or 'N/A'}\n"
        f"Employees: {d.get('number_of_employees', 'N/A')}\n"
        f"Summary:\n{d.get('summary', 'N/A')}"
    )
elif name:
    print(f"'{name}' not found in local dataset.")
else:
    print("Please enter a valid institution name.")
```